

STORED AND REFLECTED XSS ATTACKS IN DVWA

Objective:

To demonstrate Stored XSS and Reflected XSS vulnerabilities in DVWA and explain mitigation using Input Validation and Content Security Policy (CSP).

Introduction:

Cross-Site Scripting (XSS) is a web application vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users. DVWA was used to demonstrate both Stored XSS and Reflected XSS in a controlled environment.

Stored XSS Attack:

A malicious JavaScript payload was entered into the Stored XSS module. The payload was saved in the application database and executed whenever the page was viewed.

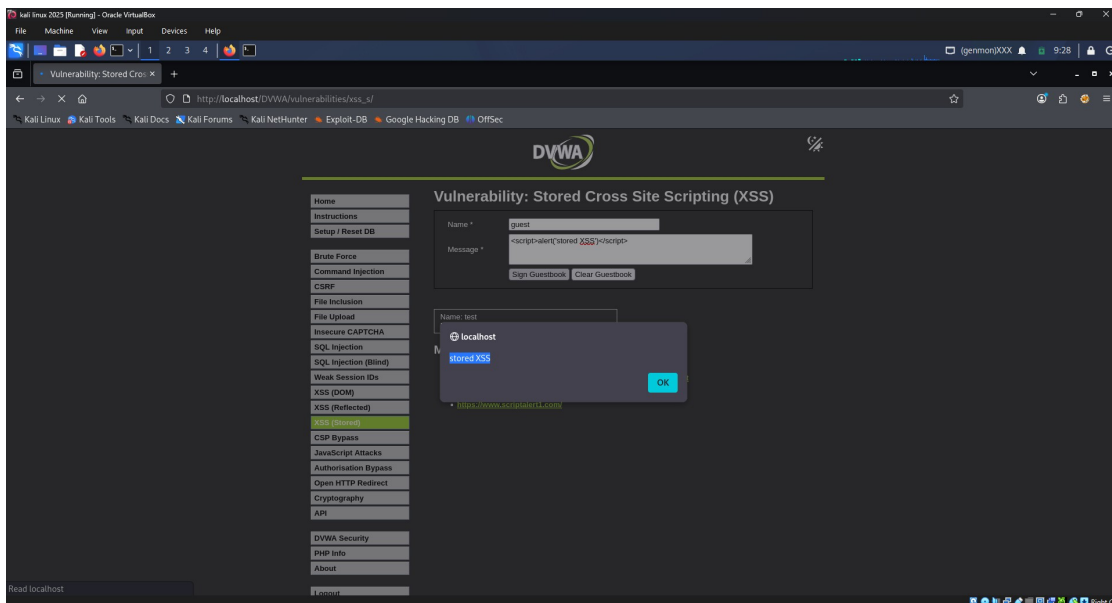


Figure 1: Stored XSS Payload Submitted in DVWA

Reflected XSS Using Query Parameters:

In the Reflected XSS module, user input was reflected back by the application without proper sanitization. A JavaScript payload was supplied through the input parameter and executed in the browser.

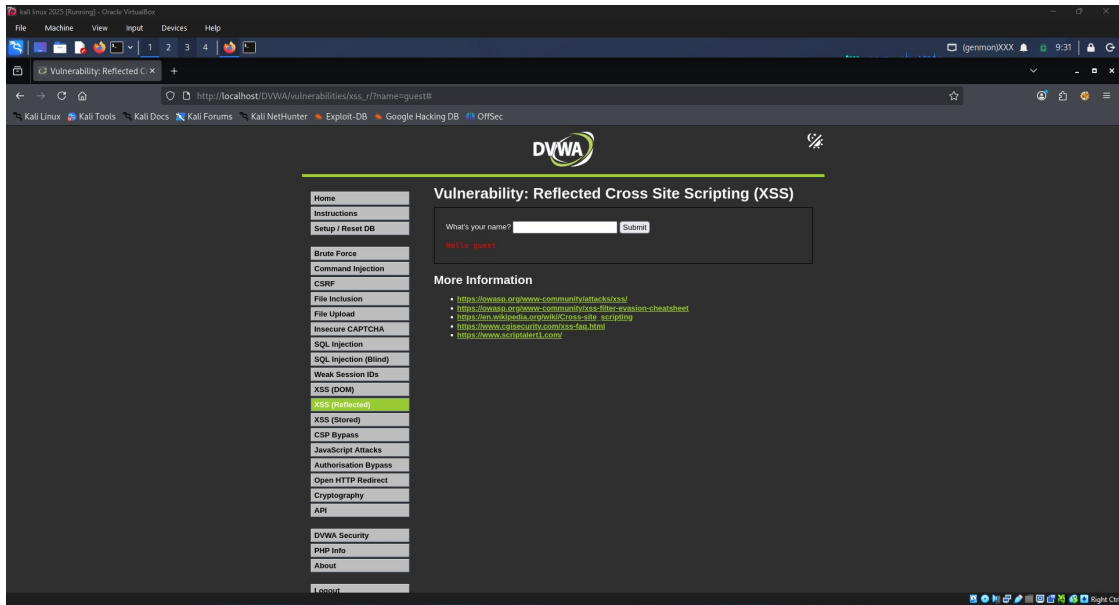


Figure 2: Reflected XSS Module - Normal Input

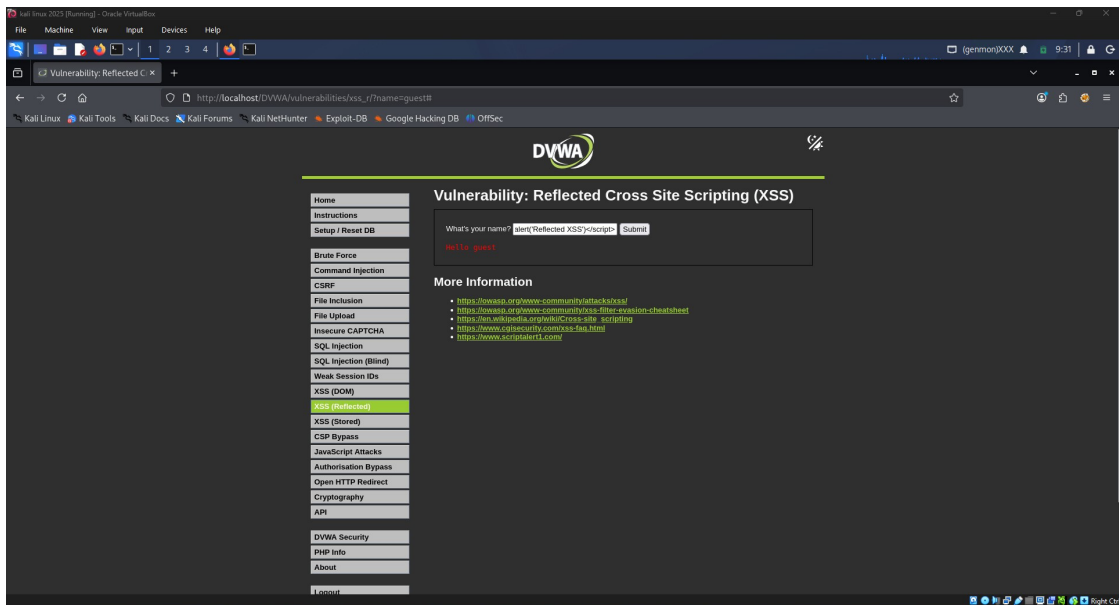


Figure 3: Reflected XSS Payload Entered

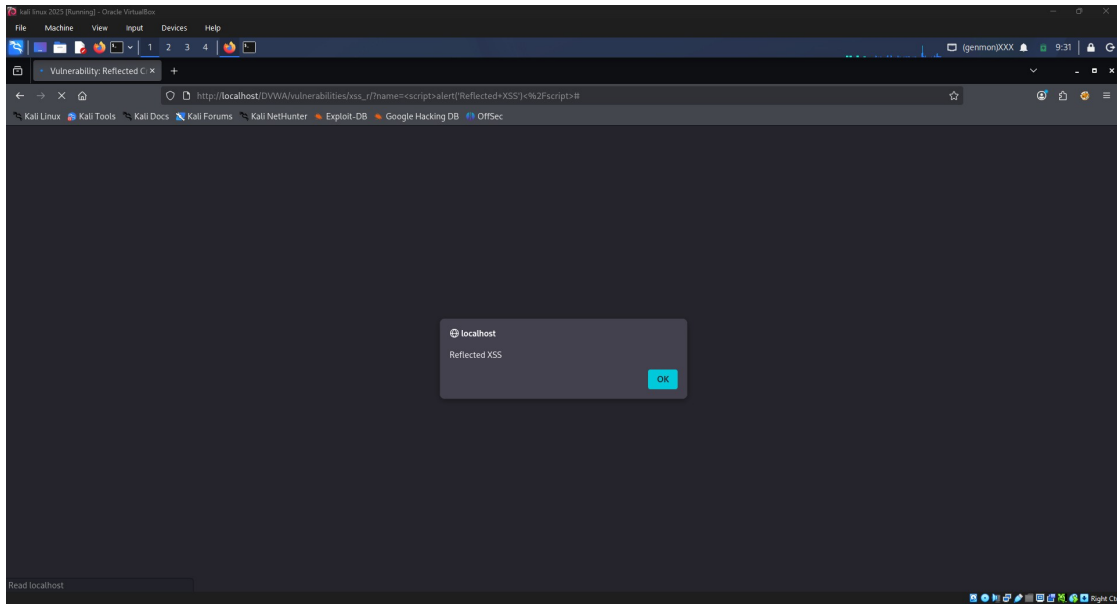


Figure 4: Reflected XSS Alert Popup Executed

Mitigation: Input Validation:

PHP Code:

```
$message = strip_tags($message);
```

This removes dangerous HTML tags such as `<script>` before storing or displaying user input. Input validation helps prevent malicious scripts from entering the application.

Mitigation: Output Encoding:

PHP Code:

```
echo htmlspecialchars($message, ENT_QUOTES, 'UTF-8');
```

This converts special characters into safe HTML entities.

For example:

```
<script>alert('XSS')</script>
```

becomes:

```
&lt;script&gt;alert('XSS')&lt;/script&gt;
```

The browser displays the text instead of executing it.

Mitigation: Content Security Policy (CSP):

Content-Security-Policy:

default-src 'self';

script-src 'self';

CSP restricts the execution of unauthorized scripts and reduces the impact of XSS attacks by allowing scripts only from trusted sources.

Conclusion:

Stored XSS and Reflected XSS vulnerabilities were successfully demonstrated in DVWA. The attacks showed how malicious scripts can execute in a user's browser. Proper input validation, output encoding, and Content Security Policy are effective controls for preventing XSS vulnerabilities.